

CS 4250

Computer Architecture

Exam 1

Prof. Nat Tuck
Plymouth State University

Fall 2026

Friday 2026-10-02 • 10:10–11:00 • D&M 417

Allowed: paper notes, pencil, a drink, a non-internet calculator. No phones. Show your work — an answer with no reasoning cannot earn partial credit.

Student Name:

1. **Ohm's law.** You have a 5 V supply and a part that wants 3.3 V at about 20 mA.
 - a. Given $V = I \cdot R$, pick a series resistor for the part. Show the calculation. If your answer isn't a stock value, pick the nearest standard resistor and say which.
 - b. How much heat (power) will that resistor emit (use)?

2. What are the names of three celestial objects in the solar system (i.e. large bodies you could see with a telescope) that are not planets?

3. Boot sequence. For the SG2000:

- a. Put these in order and say, for each, which privilege mode it runs in and what its job is:
OpenSBI, BootROM, Linux kernel, FSBL, U-Boot, init.
- b. Why can't U-Boot run directly out of the BootROM?
- c. Why does OpenSBI stay resident instead of exiting?
- d. What is the device tree (DTB) for?

4. GPIO Assembly

On the Milk-V DuoS the J3 pin-7 LED is pad VIV0_D3 = GPIO bank B channel 18 (XGPIOB_18). Its physical registers are:

Register	Address
VIV0_D3 pinmux (bits [2:0] = 3 selects GPIO)	0x03001150
SWPORTA_DDR (direction; 1 = output)	0x03021004
SWPORTA_DR (output data)	0x03021000

Bit 18 is $1 \ll 18 = 0x40000$. Here is a function that turns the LED on:

```
# void led_on(void)
led_on:
    li    t0, 0x03001150
    lw    t1, 0(t0)
    andi  t1, t1, -8        # clear mux field
    ori   t1, t1, 3         # select GPIO
    sw    t1, 0(t0)

    li    t0, 0x03021004
    lw    t1, 0(t0)
    li    t2, 0x40000
    or    t1, t1, t2
    sw    t1, 0(t0)        # pin is an output

    li    t0, 0x03021000
    sw    t2, 0(t0)        # drive pin high
    ret
```

We build this into an Arduino sketch and load it onto the little C906 core with `remoteproc`; the LED lights.

- If we build this into a C program, compile it, and run it normally as the `debian` user what will happen? Why?
- What if we run it as `root` instead. What happens? Why?
- What machine instruction would the `debian` user need to execute to control a GPIO pin? Why?

5. Dave translates a function.

Dave is translating this C function into RV64 assembly by hand:

```
long abs(long x);           /* the C library function. You cannot see its
                             source; assume it is correct and follows the
                             standard RISC-V calling convention. */

long f(long a, long b, long c) {
    return abs(a + b) + c;
}
```

His assembly:

```
f:
    addi sp, sp, -16
    sd   ra, 8(sp)
    add  a0, a0, a1        # a + b
    call abs              # a0 = abs(a + b)
    add  a0, a0, a2        # + c
    ld   ra, 8(sp)
    addi sp, sp, 16
    ret
```

Dave compiles and runs this and isn't getting correct answers on at least some inputs. He writes his own version of `abs()` and uses that instead of the library function and suddenly the program works.

- What was different about his custom `abs` function?
- How could you repair `f` so it is correct for every input, against any implementation of `abs` that follows the calling convention.